Дашборды компании

ПОКАЗАТЕЛИ СЛУЖБ АЛМИ ПАРТНЕР

Руководство по развёртыванию

Версия 0.1.0

Ноябрь 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Архитектура 4

3 Требования 5

3.1 Сервер 5

3.2 Клиенты 5

3.3 Сборка из исходного кода 5

4 Установка 6

4.1 Шаг 1. Получение дистрибутива 6

4.2 Шаг 2. Конфигурация 6

4.3 Шаг 3. Сборка и запуск 6

4.4 Шаг 4. Проверка 7

4.5 Шаг 5. HTTPS 7

4.6 Шаг 6. Первый вход 8

5 Демонстрационный стенд 9

6 Переменные окружения 10

6.1 Основные 10

6.2 Сессии и безопасность 10

6.3 Производительность 11

6.4 Журнал 12

7 База данных и миграции 13

8 Резервное копирование и восстановление 14

8.1 PostgreSQL 14

8.2 SQLite 14

9 Обновление 15

9.1 Обычное обновление 15

9.2 Обновление до 0.1.0 с версий до 0.1.0 15

10 Производительность и масштабирование 17

11 Обслуживание 18

12 Диагностика 19

1 О документе

Руководство описывает установку, настройку, обновление и обслуживание «Дашбордов компании» версии 0.1.0: состав системы, требования к серверу, запуск в Docker, переменные окружения, миграции базы данных, резервное копирование, обновление с версий до 0.1.0, проверку производительности и диагностику.

Документ предназначен для системных администраторов и инженеров эксплуатации. Работа с интерфейсом описана в «Руководстве пользователя», администрирование учётных записей и служб — в «Руководстве администратора».

Условные обозначения

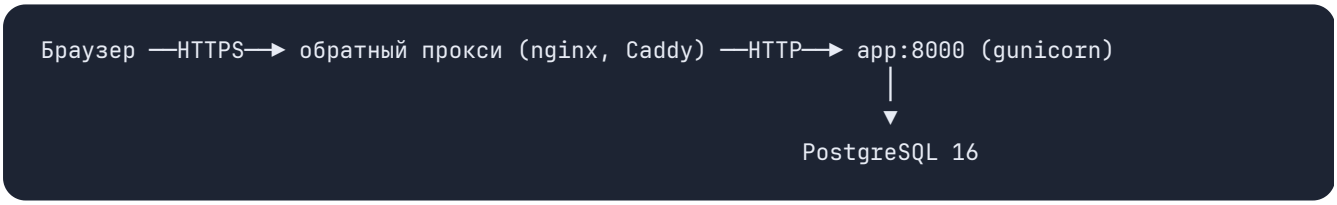
ОБОЗНАЧЕНИЕ	ЗНАЧЕНИЕ
`команда`	Команда, имя файла или переменной окружения
> **Важно.**	Правило, нарушение которого приводит к ошибке или потере данных
> **Совет.**	Рекомендация

2 Архитектура

Система состоит из двух компонентов, которые поставляются одним Docker-образом, и базы данных:

КОМПОНЕНТ	ТЕХНОЛОГИИ	НАЗНАЧЕНИЕ
Интерфейс	Vue 3, Vite, Pinia, Vue Router, Apache ECharts	Одностраничное приложение; собирается при сборке образа и раздаётся сервером
Сервер	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, gunicorn + uvicorn	API, проверка прав, расчёт аналитики, PDF-отчёты (ReportLab, Matplotlib), импорт и экспорт Excel (openpyxl)
База данных	PostgreSQL 16 (рекомендуется) или SQLite	Данные служб, пользователи, сессии, журнал изменений

Запросы обслуживают несколько рабочих процессов gunicorn с воркерами uvicorn. Состояние хранится только в базе данных (включая сессии), поэтому процессы и экземпляры приложения взаимозаменяемы: их число можно увеличивать без настройки «липких» сессий.



3 Требования

3.1 Сервер

ПАРАМЕТР	МИНИМУМ	РЕКОМЕНДУЕТСЯ ДЛЯ 500–1000 ПОЛЬЗОВАТЕЛЕЙ
Процессор	2 ядра	4 ядра и более
Память	2 ГБ	4–8 ГБ
Диск	10 ГБ	20 ГБ SSD
ОС	Linux x86_64 с Docker 24+ и Docker Compose v2	Ubuntu 24.04 LTS / Astra Linux / РЕД ОС

Отдельный сервер PostgreSQL не обязателен: `docker-compose.yml` запускает его в соседнем контейнере. Для промышленной эксплуатации можно использовать внешний кластер PostgreSQL 14+.

3.2 Клиенты

Современный браузер: Яндекс Браузер, Chrome, Edge, Firefox последних версий; экран от 1280 пикселей по ширине (на узких экранах и телефонах интерфейс перестраивается).

3.3 Сборка из исходного кода

Для разработки и сборки без Docker нужны Python 3.12+, Node.js 20.19+ (рекомендуется 22) и npm.

4 Установка

4.1 Шаг 1. Получение дистрибутива

```
git clone https://github.com/ayurasov/dashboards_serv.git
cd dashboards_serv
git checkout v0.1.0
```

4.2 Шаг 2. Конфигурация

```
cp .env.example .env
```

Откройте `.env` и задайте как минимум:

ПЕРЕМЕННАЯ	ЧТО УКАЗАТЬ
`SECRET_KEY`	Случайная строка не короче 32 символов: `openssl rand -hex 32`
`POSTGRES_PASSWORD`	Пароль пользователя базы данных
`ADMIN_PASSWORD`	Пароль первого администратора (не короче 8 символов). Если оставить пустым, временный пароль будет выведен в журнал контейнера
`APP_PORT`	Порт приложения на сервере (по умолчанию 8000)

> **Важно.** С ключом по умолчанию сервер в режиме production не запускается. Храните `.env` вне системы контроля версий.

4.3 Шаг 3. Сборка и запуск

```
docker compose up -d --build
docker compose logs -f app
```

При старте контейнер приложения:

1. ждёт готовности PostgreSQL;
2. применяет миграции базы данных (`python -m app.cli migrate`) — под блокировкой, поэтому одновременный старт нескольких контейнеров безопасен;

3. приводит справочники к актуальному виду (службы, метрики, правила светофора и палитры по умолчанию);
4. создаёт первого администратора, если пользователей в базе нет;
5. запускает gunicorn.

4.4 Шаг 4. Проверка

```
curl -s http://127.0.0.1:8000/api/health
```

Ответ `{"status": "ok", "version": "0.1.0", "database": "ok", "schema": {...}}` означает, что сервер работает, база доступна и схема актуальна. Статус `degraded` (код 503) — схема базы не совпадает с версией приложения: проверьте журнал миграций.

Откройте `http://<сервер>:8000` и войдите под учётной записью администратора. Документация API (OpenAPI) доступна по адресу `/api/docs`.

4.5 Шаг 5. HTTPS

Приложение рассчитано на работу за обратным прокси, который завершает TLS. Пример для Caddy (сертификат выпускается автоматически):

```
dashboards.company.ru {
    encode gzip
    reverse_proxy 127.0.0.1:8000
}
```

Пример для nginx:

```
server {
    listen 443 ssl http2;
    server_name dashboards.company.ru;
    ssl_certificate     /etc/ssl/dashboards.crt;
    ssl_certificate_key /etc/ssl/dashboards.key;
    client_max_body_size 25m;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }
}
```

После включения HTTPS задайте `SESSION_COOKIE_SECURE=true` и перезапустите приложение: cookie сессии будет передаваться только по защищённому соединению.

4.6 Шаг 6. Первый вход

1. Войдите под логином из `ADMIN_USERNAME` (по умолчанию `admin`) и паролем из `ADMIN_PASSWORD` или временным паролем из журнала (`docker compose logs app | grep администратор`).
2. Система попросит сменить пароль.
3. Создайте учётные записи сотрудников и выдайте доступ к службам («Руководство администратора», раздел «Пользователи»).
4. Загрузите данные служб: недельный отчёт техподдержки и выгрузку Naumen — из файлов Excel, данные службы персонала — вводом по месяцам.

5 Демонстрационный стенд

Для показа и обучения можно запустить систему с демонстрационными данными: учётные записи с разными уровнями доступа, семь месяцев данных службы персонала, 109 недель отчёта техподдержки, 1868 заявок Naumen и 142 партнёрства.

```
DEMO_DATA=true docker compose up -d --build
```

Демонстрационные данные загружаются только в пустые разделы и никогда не перезаписывают существующие. Логины и пароли демо-стенда приведены в «Руководстве администратора» (раздел «Демонстрационный стенд»).

> **Важно.** Не включайте `DEMO_DATA` на рабочем сервере: демо-пользователи получают известные пароли.

6 Переменные окружения

6.1 Основные

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`APP_ENV`	`production` в образе	`production` — строгие проверки конфигурации; `development` — для разработки
`DATABASE_URL`	задаёт compose	Адрес базы: `postgresql+psycopg://user:pass@host:5432/db` или `sqlite:////data/dashboards.db`
`SECRET_KEY`	—	Ключ подписи, не короче 32 символов
`ADMIN_USERNAME`	`admin`	Логин первого администратора
`ADMIN_PASSWORD`	пусто	Пароль первого администратора; пусто — временный пароль в журнале
`DEMO_DATA`	`false`	Загрузить демонстрационные данные в пустые разделы
`CORS_ORIGINS`	пусто	Разрешённые источники через запятую — только если интерфейс открывается с другого адреса

6.2 Сессии и безопасность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SESSION_LIFETIME_HOURS`	`12`	Максимальная длительность сессии
`SESSION_IDLE_MINUTES`	`240`	Сессия завершается после простоя
`SESSION_COOKIE_SECURE`	`false`	`true` — cookie только по HTTPS
`PASSWORD_MIN_LENGTH`	`8`	Минимальная длина пароля
`LOGIN_RATE_LIMIT`	`10`	Неудачных попыток входа с одного адреса для одного логина за окно
`LOGIN_RATE_WINDOW_SECONDS`	`300`	Окно ограничения попыток входа

6.3 Производительность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`WEB_CONCURRENCY`	2 × ядра + 1 (не больше 9)	Число рабочих процессов
`DB_POOL_SIZE`	`5`	Постоянных соединений с базой на процесс
`DB_MAX_OVERFLOW`	`4`	Дополнительных соединений на процесс при пике
`DB_POOL_TIMEOUT`	`30`	Ожидание свободного соединения, с
`DB_CONCURRENCY`	`0` — по числу соединений	Запросов, одновременно работающих с базой, в процессе; остальные ждут в очереди
`DB_QUEUE_TIMEOUT`	`30`	Ожидание в очереди к базе, с; дольше — ответ 503 «Сервер перегружен»
`THREADPOOL_SIZE`	`0` — соединения + 8	Потоков для обработчиков в процессе
`PDF_CONCURRENCY`	`2`	PDF-отчётов, одновременно строящихся в процессе
`RESPONSE_CACHE`	`true`	Кэш ответов аналитики в памяти процесса
`RESPONSE_CACHE_SIZE`	`512`	Ответов в кэше процесса
`GUNICORN_TIMEOUT`	`120`	Тайм-аут запроса, с
`GUNICORN_MAX_REQUESTS`	`2000`	Перезапуск процесса после N запросов (страховка от роста памяти)
`POSTGRES_MAX_CONNECTIONS`	`200`	`max_connections` контейнера PostgreSQL
`MAX_UPLOAD_MB`	`20`	Максимальный размер загружаемого файла

> **Совет.** Соединений с базой нужно не больше `WEB_CONCURRENCY × (DB_POOL_SIZE + DB_MAX_OVERFLOW)` на экземпляр приложения — проверьте, что `max_connections` PostgreSQL с запасом это покрывает. Значения по умолчанию (9 × 9 = 81) укладываются даже в стандартные `max_connections` = 100.

6.4 Журнал

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`LOG_LEVEL`	`INFO`	Уровень журнала
`LOG_JSON`	`false`	Журнал в формате JSON (для Loki, ELK и т. п.); пароли, токены и сессии в журнал не попадают

7 База данных и миграции

Схемой базы управляет только Alembic: приложение само таблицы не создаёт и не меняет. Миграции применяются автоматически при старте контейнера; вручную:

```
docker compose exec app python -m app.cli migrate        # применить миграции и обновить справочники
docker compose exec app python -m app.cli check-schema   # код выхода 0 — схема актуальна
```

PostgreSQL рекомендуется для всех установок. SQLite подходит для разработки и небольших стендов: в образе она работает в режиме WAL с проверкой внешних ключей.

Список таблиц и связей — в «Руководстве администратора» (раздел «Данные и хранение») и в `docs/architecture/adr-0002-database-and-migrations.md`.

8 Резервное копирование и восстановление

8.1 PostgreSQL

Резервная копия (без остановки приложения):

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > dashboards-$(date
+%F).dump
```

Восстановление:

```
docker compose stop app
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
dashboards-2026-11-01.dump
docker compose start app
```

8.2 SQLite

```
docker compose exec app python -c "import sqlite3; s=sqlite3.connect('/data/dashboards.db');
d=sqlite3.connect('/data/backup.db'); s.backup(d)"
docker compose cp app:/data/backup.db ./dashboards-$(date +%F).db
```

> **Совет.** Настройте ежедневное копирование по расписанию (cron, systemd timer) и храните копии минимум 14 дней на другом сервере. Раз в квартал проверяйте восстановление на тестовом стенде.

9 Обновление

9.1 Обычное обновление

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > before-update.dump
git fetch && git checkout vX.Y.Z
docker compose up -d --build
docker compose logs -f app
```

Миграции применятся при старте. Откат версии делается восстановлением резервной копии, сделанной перед обновлением.

9.2 Обновление до 0.1.0 с версий до 0.1.0

Версия 0.1.0 — первая версия продукта; прежние версии («Dashboards API 1.1») хранили данные в SQLite в томе `db_data`. Миграция `0012_v0_1_0_schema` перестраивает все таблицы с переносом данных: месяцы, метрики и реестр службы персонала, недельный отчёт техподдержки, заявки Naumen, партнёрства, пользователей, настройки светофора, палитры и журнал.

1. **Сделайте копию тома** со старой базой:

   ```
   docker run --rm -v dashboards_serv_db_data:/data -v "$PWD":/backup alpine cp
   /data/hr_dashboard.db /backup/hr_dashboard-before-0.1.0.db
   ```

2. **Вариант А — остаться на SQLite.** В `.env` задайте `DATABASE_URL=sqlite:////data/hr_dashboard.db` и подключите старый том к сервису `app` (раздел `volumes: - db_data:/data`). При старте база будет обновлена на месте.

3. **Вариант Б — перейти на PostgreSQL (рекомендуется).** Обновите копию старой базы локально, затем перенесите данные:

   ```
   # обновить копию SQLite до схемы 0.1.0
   docker compose run --rm -v "$PWD":/backup -e DATABASE_URL=sqlite:////backup/hr_dashboard-
   before-0.1.0.db app python -m app.cli migrate
   # создать схему в PostgreSQL и перенести данные
   docker compose up -d db
   docker compose run --rm -v "$PWD":/backup app python -m app.cli migrate
   docker compose run --rm -v "$PWD":/backup app python
   /app/backend/scripts/sqlite_to_postgres.py /backup/hr_dashboard-before-0.1.0.db
   docker compose up -d
   ```

4. **Проверьте** после входа: месяцы службы персонала, недели отчёта ТП, число заявок Naumen и партнёрств на странице «О системе».

Что меняется для пользователей при обновлении:

- роли «Начальник службы персонала» и «Просмотр отдела» превращаются в доступ к службе персонала: «Управление» и «Просмотр только своих подразделений» соответственно; роль «Просмотр» — в обычную учётную запись с её прежним доступом к службам;
- уровни доступа «чтение», «правка», «метрики», «админ» превращаются в «Просмотр», «Ввод данных», «Ввод данных», «Управление»;
- вход выполняется заново: прежние токены недействительны.

Подробности и проверки — `docs/runbooks/upgrade-to-0.1.0.md`.

10 Производительность и масштабирование

Конфигурация по умолчанию (4 ядра, `WEB_CONCURRENCY=9`, PostgreSQL в соседнем контейнере) рассчитана на 500–1000 одновременно работающих пользователей. Результаты нагрузочной проверки и методика — `docs/performance/load-test-0.1.0.md`; сценарий нагрузки — `loadtest/`.

Рекомендации:

- держите PostgreSQL на SSD; для больших объёмов заявок Naumen увеличьте `shared_buffers` до 25% памяти;
- при росте нагрузки сначала увеличивайте число процессов (`WEB_CONCURRENCY`), затем — число экземпляров приложения за балансировщиком; сессии хранятся в базе, «липкие» сессии не нужны;
- PDF-отчёты — самая тяжёлая операция (генерация графиков); при массовом формировании отчётов добавьте процессы.

11 Обслуживание

ЗАДАЧА	КОМАНДА
Сбросить пароль пользователя	`docker compose exec app python -m app.cli reset-password <логин>`
Создать администратора	`docker compose exec -e ADMIN_PASSWORD='…' app python -m app.cli create-admin <логин>`
Удалить истёкшие сессии	`docker compose exec app python -m app.cli cleanup-sessions`
Загрузить демо-данные в пустые разделы	`docker compose exec app python -m app.cli demo-data`
Проверить схему базы	`docker compose exec app python -m app.cli check-schema`

Истёкшие сессии не мешают работе (сервер их не принимает), но очистку раз в сутки удобно поставить в расписание.

12 Диагностика

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
Контейнер `app` перезапускается, в журнале «SECRET_KEY не задан»	Задайте `SECRET_KEY` не короче 32 символов в `.env`
`/api/health` отвечает `degraded`	Схема базы не актуальна: `docker compose exec app python -m app.cli migrate`, смотрите журнал
«Сессия устарела: обновите страницу»	Не совпал CSRF-токен (например, после выхода в другой вкладке) — обновите страницу
Вход сразу «выбрасывает» за HTTPS-прокси	Включён `SESSION_COOKIE_SECURE=true`, а сайт открыт по HTTP — откройте по HTTPS или выключите параметр
«Слишком много неудачных попыток»	Сработало ограничение частоты входа; подождите или перезапустите приложение
Ошибка 413 при загрузке файла	Увеличьте `MAX_UPLOAD_MB` и `client_max_body_size` прокси
Медленные страницы при большом числе пользователей	Увеличьте `WEB_CONCURRENCY`, проверьте `max_connections` PostgreSQL и загрузку процессора

Журнал приложения: `docker compose logs app`. У каждого ответа API есть заголовок `X-Request-ID` — по нему запрос находится в журнале.